

SQL NOTES

From SELECT basics to
Machine Learning magic

~ Arijit Das

• INDEX •

■ Basic SQL

- ① • Introduction to SQL.
 - Types of SQL commands (DDL, DML, DCL, TCL).
 - SQL / MySQL / PostgreSQL / SQLite.

② DDL (DATA DEFINITION LANGUAGE)

- CREATE TABLE
- ALTER TABLE
- DROP TABLE
- TRUNCATE

③ DML (DATA MANIPULATION LANGUAGE)

- INSERT INTO
- UPDATE
- DELETE FROM

④ DQL (DATA QUERY LANGUAGE)

- SELECT
- DISTINCT
- WHERE
- Logical operators ↓
(AND, OR, NOT)
- Comparison operators ↓
(>, <, =, BETWEEN, IN, LIKE)

⑤ SORTING and FILTERING

- ORDER BY
- LIMIT
- OFFSET

Intermediate SQL

(A)

① Aggregate Functions :

→ COUNT(), SUM(), MIN(), MAX(), AVG()
→ GROUP BY() [clause]

② String Functions :

→ LENGTH(), UPPER(), LOWER(), CONCAT()

③ Date Functions :

→ DAY()
→ MONTH()
→ NOW()
→ YEAR()
→ CURRENTDATE()
→ DATEDIFF()
→ TIME STAMP DIFF()

④ Mathematical Functions :

→ ROUND()
→ FLOOR()
→ CEIL()
→ MOD()

⑤ Conditional Functions :

→ IF()

⑥ Joining tables :

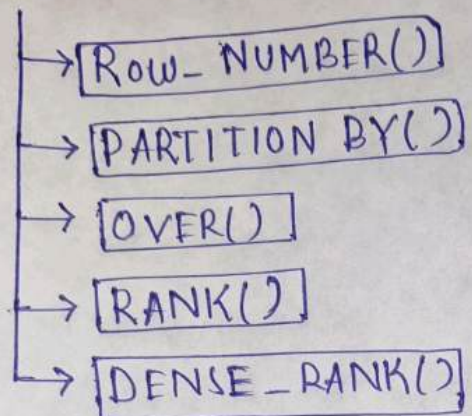
→ INNER JOIN → CROSS JOIN
→ LEFT JOIN
→ RIGHT JOIN
→ FULL JOIN
→ FULL OUTER JOIN
→ SELF JOIN

⑦ Subqueries :

→ SELECT
→ WHERE
→ FROM
→ IN
→ EXISTS
→ NON EXISTS

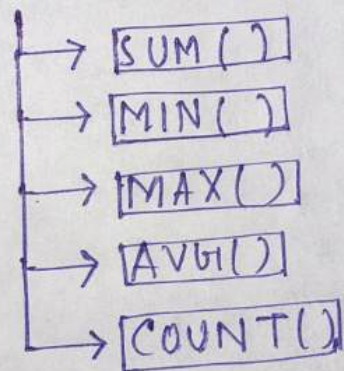
■ Advance Level

① Windows Function

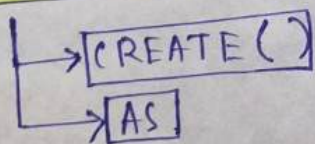


② SUBQUERIES

③ Aggregate Functions



④ VIEWS



SQL || Structure Query Language ||

* defn: SQL is a programming language used to interact with relational database.

* It is used to perform CRUD operation:

CREATE ←
READ ←
UPDATE ←
DELETE ←

CRUD

* SEQUEL (IBM)

Structured

English

Query

Language

SQL

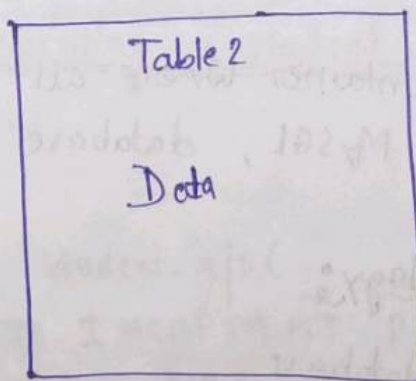
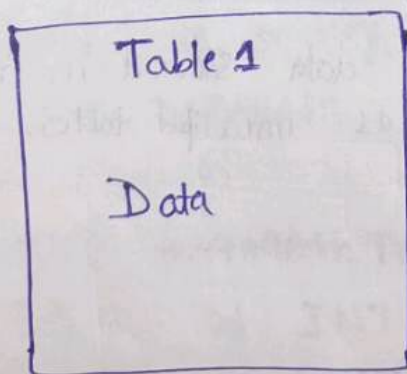
Structured

Query

Language

Database structure

Database



① What is table?

Columns →
Blueprint or
design or structure

Row →

Roll no	Name	class	DOB	Gender
1	Arijit	X	1995-06-06	M
2	Sowrabh	XII	1993-07-06	M
3	Ankit	X	1994-08-07	M
4	Rahul	XIII	1995-09-07	M

Rows → Individual
Data

↑
columns

② Creating our first database?!

CREATE DATABASE db-name;

DROP DATABASE db-name;

USE db-name;

What is database?

⇒ Database is a container where all the data stored in a organised way. In MySQL, database holds multiple tables.

• File & Folder analogy:

- Folder is a database.
- Files in the folders are the tables.
- Data in the files are called rows.

According to CRUD operation:

We can CREATE, READ, UPDATE, DELETE the structure on schema of the database.

① STEP-1 (Creating the database)

CREATE DATABASE database_name;

□ After creating the database, we have to store it or save it.

② STEP-2 (Choose which database to work in)

USE database_name;

• We use it because, we are going to creating multiple database in our system. So we have to get that unique database or specifically choose that database (Particularly).

□ After creating the database, now we have to store the tables in our database.

③ STEP-3 (Creating table inside the database)

CREATE TABLE table_name;

④ Suppose we are making or creating a student database:

CREATE DATABASE student;

USE student;

CREATE ~~DATABASE~~ TABLE student-info (

id INT AUTO INCREMENT PRIMARY KEY,

name VARCHAR(100) NOT NULL,

dept VARCHAR(100) NOT NULL,

age INT,

adhar-no CHAR(50) UNIQUE,

acc-created DATE);

id	name	dept	age	adhar-no	acc-created
NULL	NULL	NULL	NULL	NULL	NULL

↑
Table

Basically

- id, name, dept, age, adhar-no, acc-created are the columns name.
- The row showing NULL because there is no value in the rows of each column.

① Most important :

⇒ All the column name have their own datatype and Key constraints.

Syntax :

name, datatype, key constraints,

eg →

① id INT AUTO-INCREMENT PRIMARY KEY,
name data-type key constraints

② name VARCHAR (100) NOT NULL,

③ Condition

- In that table there will be one Primary key. It have to be full-filled and NOT NULL.

④

<u>Column-name</u>	<u>datatype</u>	<u>Key constraint</u>
ID	INT	PRIMARY KEY
name	VARCHAR(100) character type (100) → range	NOT NULL Because id is compulsory + enter name
dept	VARCHAR(100)	NOT NULL
age	INT	NOT NULL
adhar-no	CHAR (50)	UNIQUE
acc-created	DATE	NOT NULL

QUERYING DATA

① SELECT [According to columns]

Selecting data from a table.

① Select all columns

SELECT * FROM table-name;

② Select specific column

SELECT name, email FROM student-info;

③ RENAMING Table

RENAME TABLE student-info TO std-info;

④ To rename it back

RENAME TABLE std-info TO student-info;

② ALTER

You can use ALTER TABLE to modify existing table.

→ ADD new table column.

→ DROP that column.

→ Modify existing column's constraints.

① ADD a column

ALTER TABLE table-name ADD COLUMN column-name
DATATYPE CONSTRAINTS.

② DROP a column

ALTER TABLE table-name DROP COLUMN column-name.

③ Modify a column name.

ALTER TABLE table-name MODIFY COLUMN column-name
VARCHAR(100);

⑧ For move column, you have to mention the data type of the moving column.

iv) Move a column to the First Position.

column-name

ALTER TABLE table-name MODIFY COLUMN VARCHAR(100) FIRST;

v) Move a column after another column.

ALTER TABLE table-name MODIFY column datatype AFTER another-column.

INSERT

Inserting data into the table according to specific columns.

① Syntax 1 Best Practice INSERT USING column names.

INSERT INTO table-name (column 1, column 2, column 3, ...) VALUES
(Values of the column)

② Syntax 2 INSERT without using column names. (X No recommended)

INSERT INTO table-name VALUES
(Values of the columns)

③ INSERT multiple rows.

INSERT INTO (column 1, column 2, column 3) VALUES

(Values of each columns) ← Row 1

(Values of each columns) ← Row 2

⋮

(Values of each columns) ← Row^{nth}

Querying Data using SELECT

Basic Syntax:

Selecting columns.

```
SELECT column1, column2 FROM table-name;
```

Filtering rows with WHERE

① Equal to (=)

```
SELECT * FROM table-name WHERE column-name = 'Male';
```

```
SELECT * FROM table-name WHERE column-name = 'City';
```

```
SELECT * FROM table-name WHERE column-name = 'xyz@gmail.com';
```

② NOT EQUAL TO (!= or <>)

```
SELECT * FROM table-name WHERE column-name <> 'Kolkata';
```

```
SELECT * FROM table-name WHERE column-name != 'Kolkata';
```

③ Greater than or Equal / Less than or Equal :

```
SELECT * FROM table-name WHERE salary > 10000;
```

```
SELECT * FROM table-name WHERE salary >= 10000;
```

```
SELECT * FROM table-name WHERE salary < 10000;
```

```
SELECT * FROM table-name WHERE salary <= 10000;
```

④ IS NULL

```
SELECT * FROM table-name WHERE id IS NULL;
```

⑤ IS NOT NULL

```
SELECT * FROM table-name WHERE id IS NOT NULL;
```

⑥ BETWEEN, AND

SELECT * FROM table-name WHERE salary BETWEEN 60000 AND 80000;

⑦ IN (Used to show ^{some} specific rows by column name)

SELECT * FROM table-name WHERE salary IN (60000, 80000);

SELECT * FROM table-name WHERE id IN (1, 2);

⑧ LIKE (Pattern Matching)

SELECT * FROM table-name WHERE name LIKE 'A%';

⇒ A% -- Starts with A

⇒ %A -- Ends with A

⇒ %ji% -- contains ji

SELECT * FROM table-name WHERE name LIKE '%A';

SELECT * FROM table-name WHERE name LIKE '%A%';

⑨ AND/OR

SELECT * FROM table-name WHERE dept = 'Frontend Eng' AND salary > 60000;

SELECT * FROM table-name WHERE dept = 'Frontend Eng' OR salary < 60000;

⑩ ORDER BY (Showing the columns rows in DESC OR ASC)

SELECT * FROM table-name ORDER BY column-name DESC;

SELECT * FROM table-name ORDER BY id DESC;

SELECT * FROM table-name ORDER BY id ASC;

⊗ DESC : Descending order,

⊗ ASC : Ascending order,

LIMIT

SELECT * FROM table-name LIMIT 5;

-- shows only first 5 rows.

SELECT * FROM table-name LIMIT 10 OFFSET 5;

-- shows only the last 5 rows and skips first 5 rows.

UPDATE

■ To modify existing data using UPDATE.

Basic Syntax:

UPDATE table-name SET column 1 = Value 1 WHERE condition;

① UPDATE one specific column; and row;

UPDATE table-name SET first-name = 'Rahul' WHERE id = 1;

② UPDATE multiple column;

UPDATE table-name SET City = 'Jharkhand', email = 'corijiddas@gmail.com' WHERE id = 1;

③ Without WHERE clause; (Warning)

UPDATE table-name SET salary = 10000;

⊗ This query will affect your all rows.

DELETE

To delete specific values in a table.

■ Basic Syntax:

DELETE FROM table-name WHERE condition;

① DELETE one row:

DELETE FROM table-name
WHERE id = 1;

② DELETE multiple rows;

DELETE FROM table-name
WHERE salary < 60000;

③ DELETE all rows; (Warning)

▲ This will delete entire values which are present in those rows

DELETE FROM table-name;

④ DROP the entire table

DROP TABLE table-name;

① Aggregate Functions :

→ Count(), Sum(), min(), max(), Avg()

→ Group By

→ Having

① Count()

Count total numbers of users table.

* Syntax:

```
SELECT COUNT(*) FROM table-name;
```

① Count users who are Female;

```
SELECT COUNT(*) FROM users WHERE gender='Female';
```

Note

users = table-name

② Min()

```
SELECT MIN(salary) AS min-salary FROM users;
```

③ Max()

```
SELECT MAX(salary) AS max-salary FROM users;
```

④ Sum()

```
SELECT SUM(salary) AS total-payroll FROM users;
```

⑤ Avg()

Find average salary.

```
SELECT AVG(salary) AS average-salary FROM users;
```

⑥ Group By()

Grouping rows by category.

Average salary by gender.

Syntax

```
SELECT gender, AVG(salary) AS avg-salary FROM users  
GROUP BY gender;
```

⑧ String Functions:

→ Length()

→ upper()

→ lower()

→ concat()

① length()

length of user name.

```
SELECT username, LENGTH(username) AS len-username  
FROM users;
```

② Lower()

```
SELECT username, LOWER(username) AS lower-username  
FROM users;
```

③ Upper()

```
SELECT username, UPPER(username) AS upper-username  
FROM users;
```

④ Concat()

```
SELECT CONCAT('username', '1221') AS password  
FROM users;
```

⑤ Date Function:

① ~~DAY()~~, MONTH(), YEAR()

② NOW()

③ DATE DIFF()

④ TIME STAMP DIFF()

① ~~DAY()~~

Current date and time:

```
SELECT NOW();
```

② YEAR(), MONTH(), DAY()

Extract parts of date-of-birth:

```
SELECT name, YEAR(date-of-birth) FROM users;
```

③ DATEDIFF()

Find number of days between today and birthdate:

```
SELECT name, DATEDIFF (CURDATE(), date-of-birth) AS days-lived  
FROM users;
```

④ TIMESTAMPDIFF()

Calculate age in years:

```
SELECT name, TIMESTAMPDIFF (YEAR, date-of-birth,  
CURDATE()) AS age FROM users;
```

⑤ Mathematical Functions:

→ ROUND(), FLOOR(), CEIL()

→ MOD()

① ROUND(), FLOOR(), CEIL()

```
SELECT salary, ROUND (salary) AS rounded-salary,  
FLOOR (salary) AS floored,  
CEIL (salary) AS ceiled,
```

FROM users;

② MOD()

```
SELECT id, MOD(id, 2) AS remainder FROM users;
```

⑤ Conditional Function:

→ IF ()

```
SELECT name, IF (Dept = 'Data analyst', 'Yes', 'No')  
AS dept-data-analyst FROM users;
```

Primary Key

A primary Key is a column (or a set of column) in a table that uniquely identifies each row in that table.

Key points:

- Must be unique.
- cannot be NULL.
- Is used to identify rows in a table.
- Can be a single column or a combination of columns.
- Each table can have only one PRIMARY KEY.

Example :

```
CREATE TABLE users (  
  id INT AUTO INCREMENT PRIMARY KEY,  
);
```

* How is Primary Key Different from unique?

At first, This two might seem similar since both prevent duplicate values. But there are important difference.

Feature	<u>Primary Key</u>	<u>unique</u>
Must be unique	Yes	Yes
Allows Null values	No	Yes (one or more)
How many allowed	only one per table	Multiple.
Dropping	Cannot be easily dropped.	Can be dropped anytime.

Example of Unique:

```
id INT AUTO-INCREMENT,  
email VARCHAR(100) UNIQUE,  
Adhaar-no. INT UNIQUE);
```

Foreign Key:

A Foreign Key in SQL is a group of columns which refers to the another table's Primary Key.

A) Key Features :-

- i) A foreign key creates link between two tables.
- ii) The Primary key belongs to the Parent Table and Foreign Key belongs to the Child table.

* Most important Point: [vvi]

- When you create a child table and make a column FK (foreign key), its id no. should have to match with the Parent table Primary key column numbers.

B) Constraints:

- A foreign key accepts null values. (unless specified as NOT NULL).
- By default, if you try to insert an id number that does not match with the Parent table Primary key number, it will throw an error.

④ Action on Delete/Update:

When a referenced record in Parent table is deleted or updated, we will able control it on child table rows:

① ON DELETE CASCADE → [VVI]

Deletes matching child rows automatically.

② ON DELETE SET NULL

sets the foreign key column in child table to NULL.

③ ON DELETE RESTRICT

Prevents deletion in the Parent table if child rows exists.

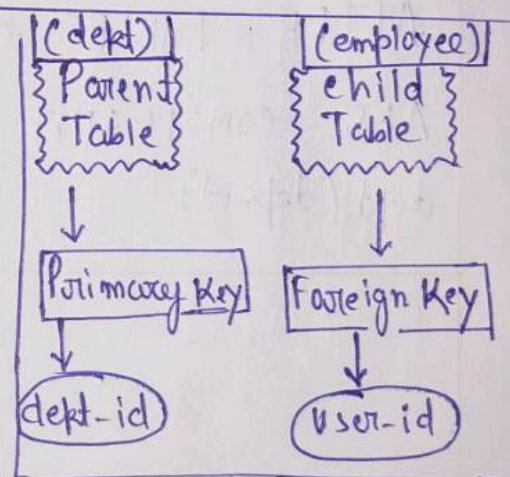
Example :

-- Parent table

```
CREATE TABLE dept (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR (100)  
);
```

-- child table

```
CREATE TABLE employee (  
    id INT PRIMARY KEY,  
    user_id INT,  
    address VARCHAR (100) NOT NULL,  
    CONSTRAINT fk-user FOREIGN KEY (user_id) REFERENCES  
    dept (dept_id) ON DELETE CASCADE  
);
```



■ Droppin a Foreign Key:

To drop a foreign key you have to know it's constraints, if you forget to make the constraint it will be auto generate key sql.

Syntax

~~ALTER TABLE~~ ~~employee~~ ~~DROP~~

ALTER TABLE employee

DROP FOREIGN KEY fk-user;

■ Adding a Foreign Key:

ALTER TABLE employee

ADD CONSTRAINT fk-user FOREIGN KEY (user-id) REFERENCES dept (dept-id)

SQL JOINS

- In SQL, JOINS are used to combine + rows of two or multiple tables based on their columns.
- Usually a foreign Key in one table referencing a Primary Key in another.

TYPES OF JOINS:

Mainly there are four types of JOINS in SQL.

→ **INNER JOIN**

→ **LEFT JOIN**

→ **RIGHT JOIN**

→ **FULL JOIN**

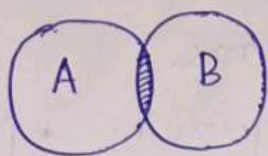
- ① If you want to see all values and their columns from both tables;

```
SELECT * FROM emp-info  
JOIN addresses ON  
emp-info.id = addresses.user-id;
```

- ② If you want to see individual selective columns from both tables;

```
SELECT emp-info.id, emp-info.name, addresses.user-id,  
addresses.street FROM emp-info INNER JOIN addresses  
ON emp-info.id = addresses.user-id;
```

④ INNER JOIN



→ Parent Table Primary Key → child Table Foreign Key

Table A			Table B	
dept-id	dept-name		emp-id	emp-name
1	BCA	✓ →	1	Arijit
2	B.Tech	✓ →	2	Arnob
3	BBA	X →	4	Vishal

Syntax

```
SELECT A.dept-id , B.emp-id
FROM A INNER JOIN B
ON A.dept-id = B.emp-id;
```

TABLE C

dept-id	emp-id
1	1
2	2

RIGHT JOIN

TABLE A

dept-id	dept-name
1	B&A
2	B.Tech
3	BBA

TABLE B

std-id	std-name
1	Aarjit
2	Aarnab
4	vishal

Syntax

SELECT A.dept-id, B.std-id
FROM A RIGHT JOIN B
ON A.dept-id = B.std-id;

TABLE C

dept-id	std-id
1	1
2	2
	4



LEFT JOIN

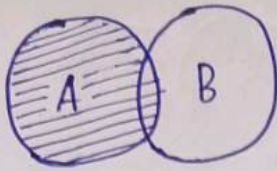


TABLE A

dept_id	dept_name
1	BEA
2	B.Tech
3	BBA

TABLE B

std_id	std_name
1	Arijid
2	Arnob
4	Vishal

Syntax

```
SELECT A.dept_id, B.std_id  
FROM A LEFT JOIN B  
ON A.dept_id = B.std_id;
```

TABLE C

dept_id	std_id
1	1
2	2
3	

FULL JOIN

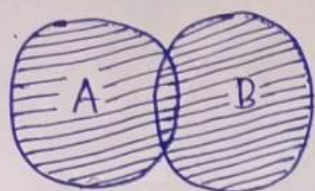


TABLE A

dept-id	dept-name
1	BEA
2	B.Tech
3	BBA

TABLE B

std-id	std-name
1	Arijit
2	Avinab
4	Vishal

Syntax

```
SELECT A.dept-id, B.std-id  
FROM A FULL JOIN B  
ON A.dept-id = B.std-id;
```

TABLE C

dept-id	std-id
1	1
2	2
3	
	4

UNION & UNION ALL

The UNION operator in SQL is used to combine two tables. It is basically use for concatenation for both tables.

Key Features:

- i) Both the tables have to be same number of columns.
- ii) To concat two tables columns, the columns data type should be same. Means the constraints (VARCHAR with VARCHAR) and (INT with INT).
- iii) It will not create any problem if the column name is different.

Table A

```
CREATE TABLE users (
```

```
    ID INT AUTO-INCREMENT PRIMARY KEY,
```

```
    Name VARCHAR(100) NOT NULL,
```

```
    email VARCHAR(100) NOT NULL,
```

```
    address VARCHAR(100) NOT NULL
```

```
);
```

```
CREATE TABLE admin-users (
```

```
    ID INT AUTO-INCREMENT PRIMARY KEY,
```

```
    First name VARCHAR(100) NOT NULL,
```

```
    Email VARCHAR(100) NOT NULL,
```

```
    Salary INT,
```

```
);
```

Syntax

```
SELECT ID, Name, email FROM users
```

```
UNION
```

```
SELECT ID, First Name, email FROM admin_users;
```

⇒ This will create two tables.

⇒ This will remove duplicate rows that contain the same name.

⇒ You have to use UNION ALL.

Syntax

```
SELECT ID, Name, email FROM users
```

```
UNION ALL
```

```
SELECT ID, First Name, email FROM admin_users;
```

⇒ You can use order by clause here :-

```
SELECT ID, Name, email, Salary FROM users
```

```
UNION ALL
```

```
SELECT ID, First Name, email, Salary FROM admin_users
```

```
ORDER BY Salary;
```

⇒ It will ascend in ascending order means smaller to greater salary.

Syntax

```
SELECT ID, Name, email FROM users
```

```
UNION
```

```
SELECT ID, First-Name, email FROM admin-users;
```

⇒ This will create two tables.

⊗ This will remove duplicate rows that contains the same name.

⇒ You have to use UNION ALL.

Syntax

```
SELECT ID, Name, email FROM Users
```

```
UNION ALL
```

```
SELECT ID, First-Name, email FROM admin-users;
```

■ You can use order by clause here

```
SELECT ID, Name, email, Salary FROM Users
```

```
UNION ALL
```

```
SELECT ID, First-Name, email, Salary FROM admin-users
```

```
ORDER BY Salary;
```

⇒ It will ascend in ascending order means smaller to greater salary.

* Important Qs *

① What is Cross-category reporting?

ms) In data analysis and reporting, cross category reporting means where you compare or analyze data.

HR context:

<u>Department</u>	<u>Male</u>	<u>Female</u>	<u>Total</u>
IT	40	15	55
HR	10	20	30
FINANCE	25	25	50

i) Employee count by dept.

ii) Employee count by dept and gender.

i) Syntax ii)

```
SELECT dept, gender, count(employee) AS total_emp  
FROM emp-info group BY dept, gender;
```

GROUP BY and HAVING

Group by:

The GROUP BY clause is used to group rows that have the same values of a particular column.

* It is typically used with aggregate functions like SUM(), COUNT(), AVG(), MIN() or MAX().

Example:

① Group the rows with same values:

i Find average salary of each department.

```
SELECT dept, avg(salary) AS 'Average salary' FROM emp-info  
GROUP BY dept;
```

ii Group all the names in the table:

```
SELECT names FROM emp-info GROUP BY name;
```

iii Find average salary of each employee in the table,

```
SELECT name, avg(salary) AS 'average salary' FROM emp-info  
GROUP BY name;
```

iv Find the referee and identify how many times he referred.

```
SELECT referred-by-id, count(referred-by-id) FROM emp-info  
GROUP BY referred-by-id;
```

Having :

After creating groups by using **GROUP BY** and also after the aggregation on a particular column, **HAVING** clause helps to filter that grouped aggregated column.

example :

- ① Find average salary > 60,000.

```
SELECT dept,
avg(salary) AS 'average salary'
FROM emp-info GROUP BY dept
HAVING avg(salary) > 60000;
```

- ② Find the referred by id who referred more than 5 times.

```
SELECT referred-by-id,
count(referred-by-id) AS 'referred-total'
FROM emp-info GROUP BY referred-by-id
HAVING count(referred-by-id) > 5;
```

Self JOIN

* A self JOIN is a regular join but the table is joined with itself.

* This is useful when the rows are in same table are related to each other.

* **Example:** Suppose you have a table and that table contains a user who refer some other users, and we want to store that person ID in the same table.

① First step 1:

You have to add a new column 'referred-by-id'.

• **Syntax**

```
ALTER TABLE emp-info ADD COLUMN referred-by-id INT;
```

② Second step 2:

Now you have to set the id of the referral who referred the users. You have to set id (referral) according to the referred users id.

• **Syntax**

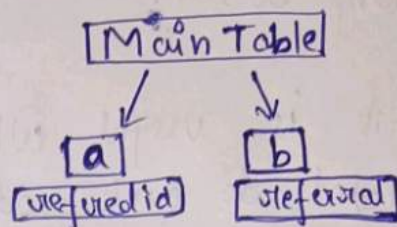
```
UPDATE emp-info SET referred-by-id = 1 WHERE ID IN (2, 4, 6, 8, 10);
```

Step-3:

- * Now there is created two tables under the main table.

→ referred-by-id / referral (b)

→ referred-id (a)



- * Now you have to select that two table. You have to use self JOINS here.

Syntax

SELECT a.id, a.First-Name AS 'referred-id',

b.id, b.First-Name AS 'referral'

FROM emp-info a

INNER JOIN

emp-info b ON

a.referred-by-id = b.id;

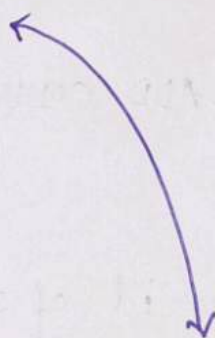


TABLE emp-info

id	First-Name	referred-by-id
1	Arijit	
2	Rahul	1
3	Kartik	
4	Arnab	1
5	Arnav	
6	Tirtha	1
7	Partha	
8	Sourav	1
9	Kunal	
10	Vishal	1

ID	referred-id	ID	referral
2	Rahul	1	Arijit
4	Arnab	1	Arijit
6	Tirtha	1	Arijit
8	Sourav	1	Arijit
10	Vishal	1	Arijit

Views

* A view in MySQL is a virtual table based on the result of SELECT query. It does not store data itself.

* It always reflects the current data in the base tables.

■ When it is useful:

i) when you want to reuse logic.

ii) Handles complex queries by not repeating them.

iii) You want a 'live snapshot' of filtered data.

■ Example:

Suppose you want a list of all users who earn more than 70,000 ₹.

```
CREATE VIEW high-salary AS
SELECT id, name, dept
FROM users
WHERE salary > 70000;
```

* The view is now created.

Now you have to call this by ↓

```
SELECT * FROM high-salary;
```

Important point

Based on the recent query ←
←

Suppose you change salary from your actual table, Automatically the created view will be changed.

Example

⇒ Step 1

```
CREATE VIEW rich-users AS  
SELECT id, name, dept  
FROM users  
WHERE Salary > 70000;
```

⇒ Step 2

```
UPDATE users SET Salary = 30000 WHERE id = 3;
```

suppose that users(3) salary was above 70000 before now you change his/her salary to 30000, now the rich-users query will be updated, it will remove that id whose salary < 70000 (30000) from that view.

⑧ You can drop a view by ↓

```
DROP VIEW rich-users;
```

Subqueries

A subquery is a query nested inside another query. Subqueries are useful for breaking down complex queries.

* Used in

→ SELECT

→ WHERE

→ FROM

■ Examples

① Salary Comparison

i Suppose you want those users who earn more than average salary.

```
SELECT id, name, dept FROM emp-info WHERE salary >
( SELECT AVG(salary) FROM emp-info );
```

ii Find users who have been referred by someone who earns more than ₹ 75 000.

```
SELECT Id, First-name FROM
emp-info WHERE referred-by-id
IN ( SELECT Id FROM emp-info WHERE salary > 75 000 );
```

iii Find users whose salary is greater than the salary of at least one employee in SDE dept.

```
SELECT Id, First-name, salary FROM emp-info WHERE
SALARY > ANY ( SELECT salary FROM emp-info WHERE
Dept = 'SDE' );
```

④ Find employees who have the same job as 'Alice'.

```
SELECT Id, name, Dept FROM
```

```
emp-info WHERE Dept IN
```

```
(SELECT Dept FROM emp-info WHERE name='Alice');
```

⑤ Find top 3 highest paying employees.

```
SELECT Id, First_name
```

```
FROM emp-info WHERE
```

```
Salary = (SELECT Salary FROM
```

```
emp-info GROUP BY Salary ORDER
```

```
BY Salary DESC LIMIT 3);
```

⑥ Find employees who earns highest salary in their dept.

```
SELECT e.name, e.id, e.dept
```

```
FROM emp-info e
```

```
WHERE e.salary = (SELECT MAX(salary)
```

```
FROM emp-info WHERE dept = e.dept);
```

* Always remember:

- Inner query is main, which runs first,
- Outer Query depends on inner query.

Stored Procedures

■ What is stored Procedures in MySQL??

→ A stored procedures are like functions in programming but written in SQL.

→ It is a set of SQL statements.

→ Instead of writing same SQL queries again and again we can save it in stored procedures and re-use it later.

→ It stored in a database (DB).

■ Why use it?

→ Reusability

→ Easy to maintain

→ Faster

→ Security

■ Syntax:

DELIMITER \$\$

CREATE PROCEDURE select_users()

BEGIN

SELECT * FROM users;

END

DELIMITER;

CALL select_users;

} It saves the query in DB

} It calls the query

- ① Create a store procedure where you can see the employees who earns maximum salary in their department.

DELIMITER \$\$

```
CREATE PROCEDURE max_salary_per_dept()  
BEGIN  
    FROM emp-info e  
    SELECT e.name, e.dept, e.location, WHERE e.salary =  
    (SELECT MAX(salary) FROM emp-info WHERE Dept = e.dept);
```

END \$\$

DEIMITER ;

CALL max_salary_per_dept ;

- We can input new row/value in our table.

• But remember some things ↓

- 1) In MySQL, the columns constraints are always defined on the table structure, not inside the stored procedure.
- 2) When you create table, you define the constraints like PRIMARY KEY, INT, VARCHAR, NOT NULL etc...
- 3) That's why you have to use the same constraints for the specific table where you are inserting data.
- 4) You can not define different constraint in the procedure.

5) In table creation, you can use constraints (NOT NULL, DEFAULT, UNIQUE).
In stored procedure parameters, you can only define the parameter name & data type. You can not write (NOT NULL, UNIQUE, DEFAULT).

Syntax:

```
DELIMITER $$  
CREATE PROCEDURE add_users(  
    p_name VARCHAR(100),  
    p_dept VARCHAR(10),  
    p_email VARCHAR(50),  
    p_salary INT,  
    p_address VARCHAR(100)  
)  
BEGIN  
    INSERT INTO emp-info (name, dept, email, salary, address)  
    VALUES (p_name, p_dept, p_email, p_salary, p_address)  
END $$  
DELIMITER ;  
  
CALL add_users('Tirtha', 'DEVOPS', 'tirtha99@gmail.com', 70000,  
'Kolkata');
```

- You can see how many stored procedures you created.

SHOW PROCEDURE STATUS WHERE DB = 'database name';

- You can drop stored procedure.

DROP PROCEDURE procedure_name;

Windows Functions

Defn

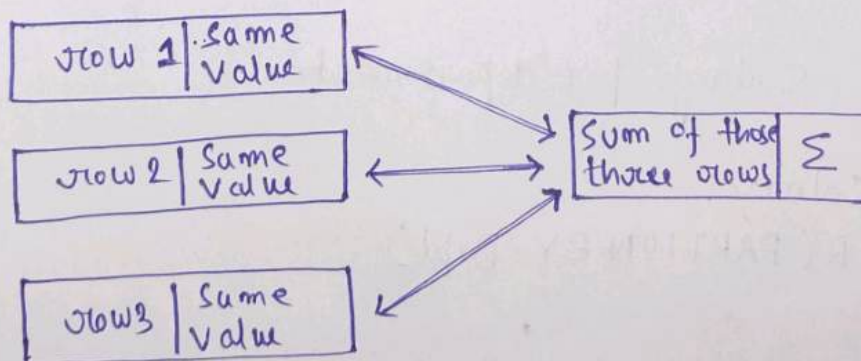
Window functions applies aggregate, ranking and analytic functions over a particular window (set of rows).

⊛ OVER clause is used with window functions to define that window.

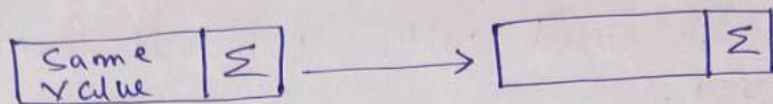
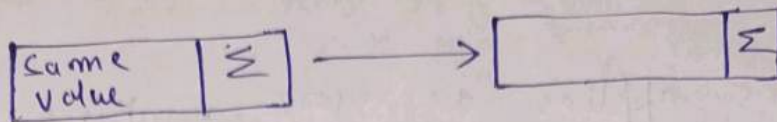
Difference between
Aggregate functions
Δ
Window functions

Work on

- In aggregate functions, a group of rows and return one result per group.
- In Window functions, do not collapse rows - they return a value for every single row.



Aggregate Functions



Window function

Syntax :

```
SELECT Column-name
Agg (Column) OVER (PARTITION BY column-name
ORDER BY column-name) AS 'column-name'
FROM table
```

Q1 Find the maximum salary per department.

```
SELECT Dept, Salary,
MAX (Salary) OVER (PARTITION BY Dept)
AS max-salary
FROM emp-info;
```

Q2 Find the average salary per department.

```
SELECT Dept, Salary,
AVG (Salary) OVER (PARTITION BY Dept)
AS avg-salary
FROM emp-info;
```

Q9 Find the minimum salary per department.

```
SELECT Dept, Salary,  
MIN(salary) OVER(PARTITION BY Dept)  
AS min-salary  
FROM emp-info;
```

■ You can use order by also. (ORDER BY, ROW-NUMBER)

```
SELECT Dept, Salary,  
ROW_NUMBER()  
OVER(PARTITION BY Dept ORDER BY salary DESC)  
AS row  
FROM emp-info
```

* First Step:-

It groups all the depts into one partition.

* Second step:-

Per partition it applies order by.

* Here in that query the all depts are divided into one partition then for each partition / group of depts it applies order by.

Q10 Get the all salaries (total) by gender.

```
SELECT gender, Salary,  
SUM(salary) OVER(PARTITION BY gender)  
AS total-salary  
FROM emp-info;
```

NOTE:

- * The row number will start from 1 and for each dept the row number will repeat again from 1.

RANK()

- **Defn:** RANK() is a window function that assigns a rank depends on the ordering of one or more columns.

- **Note:** • The duplicate row value gets the same Rank number.

■ **Syntax:**

```
SELECT dept, salary,  
RANK() OVER (PARTITION BY dept ORDER BY salary DESC)  
AS Rank-num  
FROM emp-info;
```

- Q1 Suppose you have a student table with name, class and marks. Write a query that ranks students in each class based on their marks.

```
SELECT name, class,  
RANK() OVER (PARTITION BY class ORDER BY marks DESC)  
rank-num FROM std table;
```

Q2) How would you return only the highest salary employee(s) per department using RANK()?

```
SELECT dept, salary,  
RANK() OVER (PARTITION BY dept ORDER BY salary DESC)  
AS rank_num  
FROM emp-info;
```

Q3) Given a sales table (Region, Salesman, Sales), write a query to find the 2nd highest salesperson per region.

```
SELECT Region, Salesman, Sales  
FROM (SELECT Region, Salesperson, Sales  
RANK() OVER (PARTITION BY Region ORDER BY  
Sales DESC) AS Rank_num  
FROM emp-info)  
AS t  
WHERE Rank_num = 2;
```

Q4) Given a sales table (Region, Salesman, Sales), write a query to find the 1st & 2nd highest salesperson per region.

```
SELECT * FROM (SELECT Region, Salesman, Sales,  
RANK() OVER (PARTITION BY Region ORDER BY  
Sales DESC) FROM emp-info)  
AS t  
WHERE rank_num IN (1, 2);
```

Q5 You have a table of Students (Name, class, Marks).

- Rank the students within each class based on their marks (highest marks first)
- Then return, only the highest 1 and 2 in each class.

SELECT * FROM

```
(SELECT Name, class, Marks,  
RANK() OVER (PARTITION BY class ORDER BY  
Marks DESC) AS rank_num) AS t  
WHERE rank_num IN (1,2);
```

DENSE_RANK()

Def: DENSE_RANK() is a SQL window function that gives a rank to a each row.

Key Points:

- Rows with the same values gets the same rank.
- The next rank is the very next number. (No gaps) like RANK().
- Highest value usually gets Rank 1 if order is descending

Key points:

- (i) Same value gets the same the rank.
- (ii) No skipping of ranks (After 2, next rank is 3, not 4).

Example-2 :

<u>Emp-id</u>	<u>Name</u>	<u>Dept</u>	<u>Salary</u>
1	Rahul	IT	70000
2	Sneha	IT	60000
3	Amid	IT	60000
4	Priya	HR	65000
5	Neha	HR	65000
6	Karom	HR	55000
7	Aarjun	Sales	50000

⊗ Find Salary ranking (company-wise) using DENSE-RANK().

```
SELECT Emp-id, Name, Dept, Salary  
DENSE-RANK() OVER(PARTITION BY Dept ORDER BY Salary  
DESC) AS rank-num  
FROM emp-info;
```

<u>Name</u>	<u>Dept</u>	<u>Salary</u>	<u>rank-num</u>
Rahul	IT	70000	1
Sneha	IT	60000	2
Amid	IT	60000	2
Priya	HR	65000	3
Neha	HR	65000	3
Karom	HR	55000	4
Aarjun	Sales	50000	5

<u>Name</u>	<u>class</u>	<u>Marks</u>
A	10	90
B	10	80
C	10	80
D	10	70

Query :

```
SELECT Name, class, Marks
DENSE_RANK() OVER (PARTITION BY class ORDER BY Marks
DESC) AS rank-num
FROM emp-info;
```

<u>Name</u>	<u>class</u>	<u>Marks</u>	<u>rank-num</u>
A	10	90	1
B	10	80	2
C	10	80	2
D	10	70	3

example 3 :

Find the second highest salary from every dept.

```
SELECT * FROM (SELECT Id, First-name, Dept, Salary  
DENSE_RANK() OVER (ORDER BY Salary DESC)  
AS rank-num  
FROM emp-info) AS t
```

WHERE rank-num = 2;

■ Note

- (i) Here we are choosing second-highest salary from all.
- (ii) In DENSE-RANK(), the all second highest salary will be rank=2.
- (iii) All highest salaries will be same